

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – EXPERIMENTS

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO3: Articulation (BL4, BL5)	Assessment:	Assessment Tool 1 – Experiments
		Total Marks:	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	ATHIIN SIVA.S	Reg. No.:	192524237
Section / Batch:	2025	Date:	4/8/2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Write the algorithm and execute the code for the given experiment.
- Follow a well-structured, systematic approach to design and implement an optimal solution.
- Provide insightful analysis with accurate validation, supported by clear documentation including diagrams, code, and results.

Question Type	Focus Area	Questions
Experiments	Quick Sort Performance on Nearly Sorted vs Random Datasets	Q1

SECTION A – Experiments

Evaluate the behavior of Quick Sort on nearly sorted datasets by implementing the algorithm and measuring execution time. Compare the results with random input datasets. Analyze how pivot selection impacts performance degradation. Interpret findings and justify techniques to improve efficiency.

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Experiments					
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding ; unable to integrate concepts
Experimental Design	30%	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20%	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15%	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10%	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%				
Experimental Design	30%				
Use of Tools	20%				
Analysis of Results	15%				

Documentation	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Quick Sort Performance on Nearly Sorted vs Random Datasets

Aim

To evaluate the performance of the **Quick Sort algorithm** on nearly sorted and random datasets by measuring execution time, and to analyze how different pivot-selection techniques affect efficiency.

Algorithm

1. Take an input array.
2. Select a pivot element.
3. Partition the array so that:
 - elements smaller than or equal to the pivot are placed on the left,
 - elements greater than the pivot are placed on the right.
4. Recursively apply Quick Sort to both partitions.
5. Measure the execution time.
6. Perform the experiment using:
 - nearly sorted data,
 - random data.
7. Compare different pivot-selection strategies such as the **last-element pivot** and **random pivot**.
8. Analyze the results.

Impact of Pivot Selection

The choice of pivot has a major effect on Quick Sort.

First or last element pivot: Simple to implement, but risky for sorted and nearly sorted datasets. It can cause very uneven partitions and increase recursion depth.

Random pivot: A pivot is randomly selected from the current partition. This greatly reduces the probability of repeatedly encountering worst-case partitions.

Median-of-three: The first, middle, and last elements are examined, and their median is selected as the pivot. This is particularly useful for avoiding poor pivot choices on ordered or nearly ordered data.

Conclusion

Quick Sort is an efficient sorting algorithm with an average complexity of $O(n \log n)$, but poor pivot selection can degrade its performance to $O(n^2)$. Nearly sorted datasets clearly expose this weakness when the first or last element is repeatedly selected as the pivot.

Using randomized pivot selection, median-of-three pivot selection, and hybrid sorting approaches reduces the probability of highly unbalanced partitions and improves the reliability and efficiency of Quick Sort across different types of input datasets.

CO3 AT1-EXPERIMENTS[← Back](#)Language: [Python](#) [C](#) [C++](#) [Java](#)**QUESTIONS****Q1**Quick Sort
100 marks

1/1 answered

Q1 Quick Sort

100 marks

Evaluate the behavior of Quick Sort on nearly sorted datasets by implementing the algorithm and measuring execution time. Compare the results with random input datasets. Analyze how pivot selection impacts performance degradation. Interpret findings and justify techniques to improve efficiency.

Your Code

```
1 import random
2 import time
3 import sys
4
5 sys.setrecursionlimit(20000)
6
7 def partition(arr, low, high):
8     pivot = arr[high]
9     i = low - 1
10
11     for j in range(low, high):
12         if arr[j] <= pivot:
13             i += 1
14             arr[i], arr[j] = arr[j], arr[i]
15
```

Input

stdin input

Output

```
Nearly Sorted - Last Pivot:
0.09286640002392232
Random Data - Last Pivot:
0.00267290009651331
Nearly Sorted - Random Pivot:
0.003630999497547746
```